PYTHON PROGRAMMING ACTIVITY - BASED LEARNING

10 Case Studies with Solutions

- **Name:** A.Jaideep
- **Roll Number:** 26000852
- **Branch:** Computer Science & Engineering (AI & ML)
- **Section:** CSM1
- **Course:** Python Programming
- **Institution:** Dr. RVR NRI Institute of Technology

Table of Contents

- **Case Study 1: Progressive Electricity Billing**
- **Case Study 2: Employee Payroll and Deduction Analysis**
- **Case Study 3: Multi-Plan Telecommunications Cost Optimization**
- **Case Study 4: Railway Fare Calculator**
- **Case Study 5: Retail Billing with Discount Rules**
- **Case Study 6: Manufacturing Cost and Profitability Analysis**
- **Case Study 7: Multi-Currency Travel Budget Analysis**
- **Case Study 8: Campus Event Financial Planning**
- **Case Study 9: ATM Transaction Sequence Simulation**
- **Case Study 10: Bus Passenger Occupancy Simulation**

Case Study 1: Progressive Electricity Billing System

1. Problem Statement

Design an automated processing dashboard to calculate step-progressive consumer electrical utility invoices using a tier-structured consumption scale along with standard operational line charges.

2. Objectives

Perform accurate multi-tiered scale logic computations, filter boundary anomalies safely, and systematically structure aggregate output line statements.

3. Assumptions

Units must be positive integers; tariff boundaries are strictly non-retroactive; includes a fixed connection operational overhead of INR 50.

4. Algorithm Steps

1. Initialize system parameters and receive numeric input for electricity units consumed.
2. Validate that consumption values are non-negative; branch into fault routine if violations occur.
3. Evaluate against standard block boundaries: Tier 1 (units ≤ 100), Tier 2 (units ≤ 200), Tier 3 (units > 200).
4. Apply mathematical calculation matrix corresponding to each block's exact delta weights.
5. Add base charge metric, summarize financial parameters, and push stream directly to interface presentation.

5. Structural Program Flowchart

[START] → [Input units] → {units < 0?} → (Yes: Invalid Input) / (No: Calculate Progressive Tier) → [Add Fixed Base Charge] → [END]

6. Formal Pseudocode Specification

```
FUNCTION CalculateElectricityBill(units)
  IF units < 0 THEN RETURN 'Invalid input'
  IF units <= 100 THEN bill = units * 2
  ELSE IF units <= 200 THEN bill = 100 * 2 + (units - 100) * 3
  ELSE bill = 100 * 2 + 100 * 3 + (units - 200) * 5
  total = bill + 50
  RETURN total
END FUNCTION
```

7. Python Implementation Script

```
units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid input")
else:
    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 100 * 2 + (units - 100) * 3
    else:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5
    total = bill + 50
    print("Electricity Bill =", total)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
-45	Invalid input	Pass
80	Electricity Bill = 210	Pass
150	Electricity Bill = 400	Pass
250	Electricity Bill = 800	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter units consumed: 250
[CONSOLE OUT] Electricity Bill = 800
```

10. Analytical Conclusion

The conditional tier processing engine scales flawlessly across complex boundary jumps without data leakages or processing overflows.

Case Study 2: Employee Payroll and Deduction Analysis

1. Problem Statement

Formulate a localized salary accounting processing pipeline to handle base salary structures, static allowances, variable overtime rates, and tax deduction metrics.

2. Objectives

Perform high-precision calculations for House Rent Allowance (HRA) at 20%, general allowances at 10%, scale overtime shifts linearly at INR 150/hr, and parse standardized gross tax deductions at 8%.

3. Assumptions

Base parameters cannot fall beneath absolute zero thresholds; overtime hourly metrics are stored as discrete structural items.

4. Algorithm Steps

1. Accept input basic salary and input overtime hours.
2. Calculate structural constants: HRA = 20% of basic, general allowances = 10% of basic.
3. Compute overtime pay by multiplying hours by 150.
4. Establish gross aggregate = basic + HRA + allowance + overtime_pay.
5. Deduct statutory 8% off gross amount to compute net payout fields.

5. Structural Program Flowchart

[START] → [Input salary & overtime hours] → [Compute HRA, Allowances, Overtime pay] → [Gross = Sum of all credits] → [Net = Gross - 8% Deductions] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT basic, overtime_hours
  hra = basic * 0.20
  allowance = basic * 0.10
  overtime_pay = overtime_hours * 150
  gross = basic + hra + allowance + overtime_pay
  deduction = gross * 0.08
  net = gross - deduction
  PRINT gross, net
END
```

7. Python Implementation Script

```
basic = float(input("Enter basic salary: "))
overtime_hours = int(input("Enter overtime hours: "))
hra = basic * 0.20
allowance = basic * 0.10
overtime_pay = overtime_hours * 150
gross = basic + hra + allowance + overtime_pay
deduction = gross * 0.08
net = gross - deduction
print("Gross Salary =", gross)
print("Net Salary =", net)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
50000, 10	Gross = 66500; Net = 61180	Pass
30000, 0	Gross = 39000; Net = 35880	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter basic salary: 50000
[CONSOLE OUT] Enter overtime hours: 10
[CONSOLE OUT] Gross Salary = 66500.0
[CONSOLE OUT] Net Salary = 61180.0
```

10. Analytical Conclusion

The payroll matrix standardizes statutory adjustments safely with high precision and handles overtime calculations linearly without exception errors.

Case Study 3: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Construct a dynamic recommendation framework that evaluates a user's data consumption data matrix against multiple network provider plans to identify the cheapest plan.

2. Objectives

Compare three plan algorithms simultaneously and select the absolute minimal financial constraint path safely.

3. Assumptions

Data consumption values are real metrics expressed in Gigabytes (GB). Base costs are static flat fees.

4. Algorithm Steps

1. Capture expected consumption data footprint in GB from user.
2. Evaluate cost metrics for Plan A: Base 199, premium 50/GB over 2GB boundary.
3. Evaluate cost metrics for Plan B: Base 299, premium 30/GB over 5GB boundary.
4. Evaluate cost metrics for Plan C: Base 399, premium 20/GB over 10GB boundary.
5. Compare results via a minimum tracking algorithm and print the best plan classification identifier.

5. Structural Program Flowchart

[START] → [Input data usage (GB)] → [Compute formulas for Plans A, B, C] → [Determine minimum cost plan] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT usage
  A = 199 + MAX(0, usage - 2) * 50
  B = 299 + MAX(0, usage - 5) * 30
  C = 399 + MAX(0, usage - 10) * 20
  best = MIN(A, B, C)
  PRINT plan details and choice
END
```

7. Python Implementation Script

```
usage = float(input("Enter data usage (GB): "))
planA = 199 + max(0, usage - 2) * 50
planB = 299 + max(0, usage - 5) * 30
planC = 399 + max(0, usage - 10) * 20
print("Plan A =", planA)
print("Plan B =", planB)
print("Plan C =", planC)
minimum = min(planA, planB, planC)
if minimum == planA:
    print("Best Plan: A")
elif minimum == planB:
    print("Best Plan: B")
else:
    print("Best Plan: C")
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
1	Plan A=199, B=299, C=399 -> Best: A	Pass
6	Plan A=399, B=329, C=399 -> Best: B	Pass
15	Plan A=849, B=599, C=499 -> Best: C	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter data usage (GB): 6
[CONSOLE OUT] Plan A = 399.0
[CONSOLE OUT] Plan B = 329.0
[CONSOLE OUT] Plan C = 399.0
[CONSOLE OUT] Best Plan: B
```

10. Analytical Conclusion

The multi-plan optimization model successfully calculates dynamic break-even cross points across different data consumption ranges.

Case Study 4: Railway Fare Calculator with Demographic Rules

1. Problem Statement

Build a mass transit routing pricing system based on traveled rail distance, travel tier structural classes, and age demographic concession metrics.

2. Objectives

Enforce dynamic ticket pricing matrix, implement absolute complete discount concessions for infants (< 5 years), and apply 25% price breaks for senior citizens (>= 60 years).

3. Assumptions

Base multipliers are 1.5 for Sleeper, 2.5 for AC3, and 4.0 for AC1 class.

4. Algorithm Steps

1. Ingest user metrics for distance, age, and passenger travel class category choice.
2. Map conditional class choice directly to corresponding rate scale factors.
3. Compute initial raw fare = distance * mapped_rate.
4. Evaluate age logic: if under 5, drop fare to zero; if 60 or above, multiply fare by 0.75.
5. Transmit output ticket fare statement to the console.

5. Structural Program Flowchart

[START] → [Input distance, age, class] → [Map class to base rate] → {Age condition check} → [Apply concession adjustments] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT distance, age, travel_class
  IF travel_class == 1 THEN rate = 1.5
  ELSE IF travel_class == 2 THEN rate = 2.5
  ELSE rate = 4
  fare = distance * rate
  IF age < 5 THEN fare = 0
  ELSE IF age >= 60 THEN fare = fare * 0.75
  PRINT fare
END
```

7. Python Implementation Script

```
distance = float(input("Enter distance: "))
age = int(input("Enter age: "))
travel_class = int(input("1-Sleeper 2-AC3 3-AC1: "))
if travel_class == 1:
    rate = 1.5
elif travel_class == 2:
    rate = 2.5
else:
    rate = 4
fare = distance * rate
if age < 5:
    fare = 0
elif age >= 60:
    fare = fare * 0.75
print("Final Fare =", fare)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
100, 4, 2	Final Fare = 0.0	Pass
200, 65, 3	Final Fare = 600.0	Pass
100, 30, 1	Final Fare = 150.0	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter distance: 200
[CONSOLE OUT] Enter age: 65
[CONSOLE OUT] 1-Sleeper 2-AC3 3-AC1: 3
[CONSOLE OUT] Final Fare = 600.0
```

10. Analytical Conclusion

Demographic concession filters wrap correctly over class-specific baseline matrix multipliers without precision loss.

Case Study 5: Retail Billing with Multi-Layered Discount Rules

1. Problem Statement

Develop a checkout processing core that tracks bulk transactional amounts, tallies item matrix arrays, and applies layered stackable promotional rules dynamically.

2. Objectives

Iterate sequentially through purchase items, apply a 5% high-volume order discount for bills over INR 5000, and stack an extra 10% discount for premium loyalty cardholders.

3. Assumptions

User processes exactly 3 distinct item units sequentially; discounts stack additively based on the gross volume base.

4. Algorithm Steps

1. Loop through an iteration constraint of 3 cycles to capture prices and quantities.
2. Accumulate aggregate bill gross total inside loop logic.
3. Query string condition state for premium membership validation.
4. Apply volume rule (5% if total ≥ 5000) and membership rule (10% if affirmative).
5. Subtract compound deductions and present final metrics.

5. Structural Program Flowchart

[START] → [Loop 3 times for prices/quantities] → [Accumulate gross total] → {Check volume & loyalty criteria} → [Deduct compound discounts] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  total = 0
  FOR i = 1 TO 3
    INPUT price, qty
    total = total + (price * qty)
  ENDFOR
  INPUT member
  discount = 0
  IF total >= 5000 THEN discount = discount + total * 0.05
  IF member == 'yes' THEN discount = discount + total * 0.10
  net = total - discount
  PRINT total, discount, net
END
```

7. Python Implementation Script

```
total = 0
for i in range(1, 4):
    price = float(input(f"Enter price of item {i}: "))
    qty = int(input(f"Enter quantity of item {i}: "))
    total += price * qty
member = input("Premium member (yes/no): ")
discount = 0
if total >= 5000:
    discount += total * 0.05
if member.lower() == "yes":
    discount += total * 0.10
net = total - discount
print("Total Bill =", total)
print("Discount =", discount)
print("Net Amount =", net)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
2000,2, 1000,1, 500,1, yes	Total=5500, Disc=825, Net=4675	Pass
100,2, 200,1, 100,1, no	Total=500, Disc=0, Net=500	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter price of item 1: 2000
[CONSOLE OUT] Enter quantity of item 1: 2
[CONSOLE OUT] Premium member (yes/no): yes
[CONSOLE OUT] Total Bill = 5500.0
[CONSOLE OUT] Net Amount = 4675.0
```

10. Analytical Conclusion

The multi-tiered discount processing module handles stacked conditional streams seamlessly, preventing logic tracking race faults.

Case Study 6: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Formulate a manufacturing plant inventory pipeline to resolve full production overhead expenses and determine unit cost pricing based on profit margin targets.

2. Objectives

Aggregate structural components (materials, labor, overheads), evaluate fixed factory profit targets at 20%, and yield clean unit selling prices.

3. Assumptions

Production unit volumes are non-zero positive structural numeric inputs.

4. Algorithm Steps

1. Capture raw metrics for material cost, labor cost, and overhead expenses.
2. Aggregate components to resolve total production cost landscape.
3. Compute factory operational profit parameter at a flat 20% tier multiplier.
4. Divide cumulative cost and profit allocation by total volume units.
5. Display itemized cost breakdowns and unit-level target pricing profiles.

5. Structural Program Flowchart

[START] → [Input material, labor, overheads, units] → [Total Cost = Sum] → [Profit = Total Cost * 20%] → [Selling Price = (Total Cost + Profit) / Units] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT material, labor, overhead, units
  total_cost = material + labor + overhead
  profit = total_cost * 0.20
  selling_price = (total_cost + profit) / units
  PRINT total_cost, profit, selling_price
END
```

7. Python Implementation Script

```
material = float(input("Material Cost: "))
labor = float(input("Labor Cost: "))
overhead = float(input("Overhead Cost: "))
units = int(input("Units Produced: "))
total_cost = material + labor + overhead
profit = total_cost * 0.20
selling_price = (total_cost + profit) / units
print("Total Cost =", total_cost)
print("Profit =", profit)
print("Selling Price per Unit =", selling_price)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
10000, 5000, 5000, 1000	Total=20000, Profit=4000, UnitPrice=24.0	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Material Cost: 10000
[CONSOLE OUT] Labor Cost: 5000
[CONSOLE OUT] Units Produced: 1000
[CONSOLE OUT] Selling Price per Unit = 24.0
```

10. Analytical Conclusion

The cost analysis ledger provides structural transparency into asset distributions and target unit yields.

Case Study 7: Multi-Currency Travel Budget Analysis

1. Problem Statement

Construct a real-time international forex validation system to audit domestic travel ledger allocations against global currency transaction updates.

2. Objectives

Convert domestic currency fields using baseline exchange factors (1 USD = 83 INR) and track compliance limitations securely.

3. Assumptions

Exchange parameter ratios are locked at a static evaluation index of 83.0.

4. Algorithm Steps

1. Capture total native budget tracking limits in INR and foreign expenses in USD.
2. Normalize native boundaries by dividing by the current exchange rate index (83).
3. Perform condition limit evaluations: verify if expenses fall within normalized caps.
4. Log clear operational system messages outlining budget compliance status.

5. Structural Program Flowchart

[START] → [Input INR budget & USD expenses] → [Convert INR to USD / 83] → {USD Expenses ≤ Budget?} → [Log compliance status] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT budget_inr, expense_usd
  usd_budget = budget_inr / 83
  IF expense_usd <= usd_budget THEN
    PRINT 'Within Budget'
  ELSE
    PRINT 'Budget Exceeded'
  ENDIF
END
```

7. Python Implementation Script

```
budget = float(input("Budget in INR: "))
expense = float(input("Expense in USD: "))
usd_budget = budget / 83
print("Budget in USD =", round(usd_budget, 2))
if expense <= usd_budget:
    print("Within Budget")
else:
    print("Budget Exceeded")
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
83000, 900	Budget USD=1000.0 -> Within Budget	Pass
83000, 1200	Budget USD=1000.0 -> Budget Exceeded	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Budget in INR: 83000
[CONSOLE OUT] Expense in USD: 1200
[CONSOLE OUT] Budget in USD = 1000.0
[CONSOLE OUT] Budget Exceeded
```

10. Analytical Conclusion

The forex calculation layer standardizes external variables safely without boundary overflow faults.

Case Study 8: Campus Event Financial Planning Ledger

1. Problem Statement

Formulate a consolidated ledger dashboard to audit institutional event costs, aggregate ticket sales channels, and evaluate external corporate sponsorship capital.

2. Objectives

Synthesize multiple inbound cash streams and map raw operational costs against them to find the true financial balance.

3. Assumptions

Financial ledger elements are parsed as real decimal values.

4. Algorithm Steps

1. Capture raw metrics for tickets sold, ticket unit price, sponsorship injects, and structural event expenses.
2. Compute cumulative gross system revenue = (tickets * price) + sponsor.
3. Determine operational delta margin = revenue - expenses.
4. Route through categorical checks to log Profit, Loss, or Break-Even status.

5. Structural Program Flowchart

[START] → [Input tickets, price, sponsor, expenses] → [Revenue = Tickets * Price + Sponsor] → [Net = Revenue - Expenses] → {Evaluate Net balance} → [END]

6. Formal Pseudocode Specification

```
BEGIN
  INPUT tickets, price, sponsor, expenses
  revenue = tickets * price + sponsor
  net = revenue - expenses
  IF net > 0 THEN PRINT 'Profit', net
  ELSE IF net < 0 THEN PRINT 'Loss', ABS(net)
  ELSE PRINT 'Break Even'
END
```

7. Python Implementation Script

```
tickets = int(input("Tickets Sold: "))
price = float(input("Ticket Price: "))
sponsor = float(input("Sponsorship Amount: "))
expenses = float(input("Expenses: "))
revenue = tickets * price + sponsor
net = revenue - expenses
print("Revenue =", revenue)
if net > 0:
    print("Profit =", net)
elif net < 0:
    print("Loss =", abs(net))
else:
    print("Break Even")
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
500, 100, 5000, 40000	Revenue=55000, Profit=15000	Pass
100, 50, 0, 10000	Revenue=5000, Loss=5000	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Tickets Sold: 500
[CONSOLE OUT] Revenue = 55000.0
[CONSOLE OUT] Profit = 15000.0
```

10. Analytical Conclusion

The financial tracking logic balances multiple distinct revenue streams securely against centralized cost pools.

Case Study 9: ATM Transaction Sequence Simulation Framework

1. Problem Statement

Design an online state controller to simulate retail ATM transactions including real-time deposit adjustments and withdrawal validations.

2. Objectives

Maintain accurate financial data, prevent over-withdrawal errors, and output real-time validation confirmations safely.

3. Assumptions

Base baseline balances open at INR 1000 precisely; negative value submittals are handled upstream.

4. Algorithm Steps

1. Establish core baseline account balance state structural index at 1000.
2. Capture numeric variables for desired deposit amount and intended withdrawal run.
3. Commit deposit adjustments directly to balance tracker variables.
4. Verify if the withdrawal amount is less than or equal to current balances.
5. Update state values on success or throw standard isolation alerts on failure.

5. Structural Program Flowchart

[START] → [Initialize Balance = 1000] → [Input deposit & withdrawal] → [Balance += Deposit] → {Withdrawal ≤ Balance?} → [Execute deduction / error] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  balance = 1000
  INPUT deposit, withdraw
  balance = balance + deposit
  IF withdraw <= balance THEN
    balance = balance - withdraw
    PRINT 'Withdrawal Successful'
  ELSE
    PRINT 'Insufficient Balance'
  ENDIF
  PRINT balance
END
```

7. Python Implementation Script

```
balance = 1000
total = 0 # placeholder
deposit = int(input("Enter deposit amount: "))
withdraw = int(input("Enter withdrawal amount: "))
balance = balance + deposit
if withdraw <= balance:
    balance = balance - withdraw
    print("Withdrawal Successful")
else:
    print("Insufficient Balance")
print("Final Balance =", balance)
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
500, 200	Withdrawal Successful; Balance=1300	Pass
0, 2000	Insufficient Balance; Balance=1000	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Enter deposit amount: 500
[CONSOLE OUT] Enter withdrawal amount: 200
[CONSOLE OUT] Withdrawal Successful
[CONSOLE OUT] Final Balance = 1300
```

10. Analytical Conclusion

The state transaction processor protects against over-withdrawal vulnerabilities, maintaining system balance integrity.

Case Study 10: Bus Passenger Occupancy Simulation System

1. Problem Statement

Develop a real-time transit telemetry simulator to monitor mass transit passenger volume levels against strict maximum vehicle capacity guidelines.

2. Objectives

Calculate net passenger counts dynamically and trigger safety alerts if the physical limit is exceeded.

3. Assumptions

Maximum safety threshold is limited to 50 passengers.

4. Algorithm Steps

1. Establish fixed vehicle safety capacity ceiling benchmark at 50 units.
2. Read operational transit metrics for boarding passengers and leaving passengers.
3. Evaluate the current onboard occupant load.
4. Compare current load against safety ceiling parameters.
5. Output standard clearance records or capacity breach alerts.

5. Structural Program Flowchart

[START] → [Set Capacity = 50] → [Input boarding & leaving counts] → [Passengers = Boarding - Leaving] → {Passengers > Capacity?} → [Trigger safety threshold flag / status] → [END]

6. Formal Pseudocode Specification

```
BEGIN
  capacity = 50
  INPUT boarding, leaving
  passengers = boarding - leaving
  PRINT passengers
  IF passengers > capacity THEN
    PRINT 'Bus Capacity Exceeded'
  ELSE
    PRINT 'Within Capacity'
  ENDIF
END
```

7. Python Implementation Script

```
capacity = 50
boarding = int(input("Passengers boarding: "))
leaving = int(input("Passengers leaving: "))
passengers = boarding - leaving
print("Passengers in Bus =", passengers)
if passengers > capacity:
    print("Bus Capacity Exceeded")
else:
    print("Within Capacity")
```

8. Robust Verification Test Suite Matrix

Input Scenarios	Expected Output Footprint	Status
60, 5	Passengers=55 -> Bus Capacity Exceeded	Pass
30, 10	Passengers=20 -> Within Capacity	Pass

9. Simulated Console Execution Trace Log

```
[CONSOLE OUT] Passengers boarding: 60
[CONSOLE OUT] Passengers leaving: 5
[CONSOLE OUT] Passengers in Bus = 55
[CONSOLE OUT] Bus Capacity Exceeded
```

10. Analytical Conclusion

The capacity tracking module flags spatial safety margins reliably, preventing transit over-occupancy conditions.